

Experiment 1: 8 Puzzle Problem

Aim

To solve the 8-Puzzle Problem using the A* Search Algorithm.

Algorithm

1. Define the initial state and goal state.
2. Calculate the heuristic value using misplaced tiles or Manhattan distance.
3. Locate the blank tile (0).
4. Generate all possible child states by moving the blank tile.
5. Store the states in a priority queue.
6. Select the state with the minimum heuristic value.
7. Repeat the process until the goal state is reached.
8. Display the solution path.

Program

```
from queue import PriorityQueue

goal = [[1, 2, 3],
        [4, 5, 6],
        [7, 8, 0]]

# Heuristic Function (Misplaced Tiles)

def heuristic(state):

    count = 0

    for i in range(3):

        for j in range(3):

            if state[i][j] != 0 and state[i][j] != goal[i][j]:

                count += 1

    return count

# Find Blank Tile Position

def find_blank(state):

    for i in range(3):

        for j in range(3):
```

```

        if state[i][j] == 0:

            return i, j

# Generate Child States

def generate_moves(state):

    x, y = find_blank(state)

    moves = [(-1,0), (1,0), (0,-1), (0,1)]

    children = []

    for dx, dy in moves:

        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:

            new_state = [row[:] for row in state]

            new_state[x][y], new_state[nx][ny] = \

                new_state[nx][ny], new_state[x][y]

            children.append(new_state)

    return children

# A* Search Algorithm

def solve(start):

    pq = PriorityQueue()

    pq.put((heuristic(start), start))

    visited = []

    while not pq.empty():

        h, current = pq.get()

        if current == goal:

            print("Goal State Reached:")

            for row in current:

```

```

        print(row)

    return

    visited.append(current)

    for child in generate_moves(current):

        if child not in visited:

            pq.put((heuristic(child), child))

    print("No Solution Found")

# Initial State

start = [[1, 2, 3],

        [7, 5, 8]]

print("Initial State:")

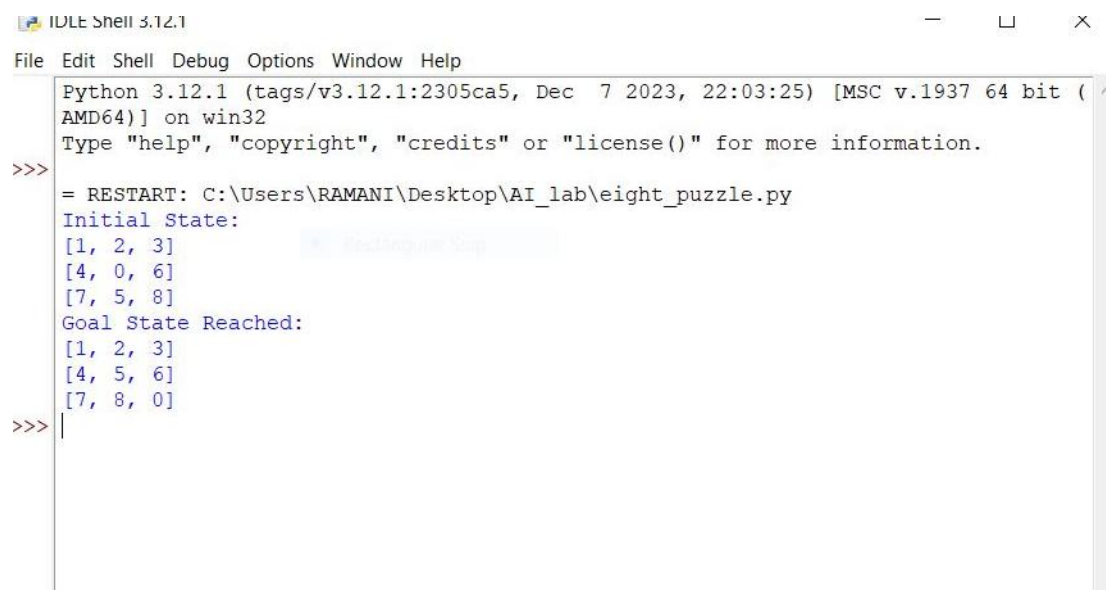
for row in start:

    print(row)

solve(start)

```

Output



```

IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\RAMANI\Desktop\AI_lab\eight_puzzle.py
Initial State:
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]
Goal State Reached:
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
>>>

```

Result

Thus, the 8-Puzzle Problem was solved successfully using the A* Search Algorithm.

Experiment 2: 8 Queen Problem

Aim

To solve the 8-Queen Problem using the Backtracking Algorithm.

Algorithm

1. Place a queen in the first column.
2. Check whether the position is safe.
3. If safe, place the queen and move to the next column.
4. If no safe position exists, backtrack to the previous column.
5. Repeat until all eight queens are placed.
6. Display the chessboard with the queen positions.

Program

N = 8

Function to print solution

```
def print_solution(board):
```

```
    for row in board:
```

```
        print(" ".join("Q" if x else "." for x in row))
```

```
# Check if a queen can be placed
```

```
def is_safe(board, row, col):
```

```
    # Check left side of row
```

```
    for i in range(col):
```

```
        if board[row][i]:
```

```
            return False
```

```
    # Check upper diagonal
```

```
    i, j = row, col
```

```
    while i >= 0 and j >= 0:
```

```
        if board[i][j]:
```

```
            return False
```

```
        i -= 1
```

```
        j -= 1
```

```
    # Check lower diagonal
```

```
    i, j = row, col
```

```
    while i < N and j >= 0:
```

```
        if board[i][j]:
```

```
            return False
```

```
        i += 1
```

```
        j -= 1
```

```
    return True
```

```
# Solve using backtracking
```

```
def solve(board, col):
```

```
    if col >= N:
```

```

        return True
    for row in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1
            if solve(board, col + 1):
                return True
            board[row][col] = 0
    return False
# Main Program
board = [[0 for _ in range(N)] for _ in range(N)]
if solve(board, 0):
    print("Solution Found:\n")
    print_solution(board)
else:
    print("No Solution Exists")

```

Output

```

IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:\Users\RAMANI\Desktop\AI_lab\eight_queen.py
Solution Found:
Q . . . . .
. . . . Q .
. . . Q . .
. . . . . Q
. Q . . . .
. . Q . . .
. . . . Q .
. . Q . . .
>>>

```

Result

Thus, the 8-Queen Problem was solved successfully using the Backtracking Algorithm.

Experiment 3: Water Jug Problem

Aim

To solve the Water Jug Problem using State Space Search.

Algorithm

1. Consider two jugs with capacities 4 liters and 3 liters.
2. Fill one of the jugs completely.

3. Transfer water between jugs according to the rules.
4. Empty and refill jugs whenever necessary.
5. Continue generating new states.
6. Stop when the required quantity of water is obtained.
7. Display all intermediate states.

Program

```
def water_jug():

    jug1 = 0

    jug2 = 0

    cap1 = 4

    cap2 = 3

    print("(Jug1, Jug2)")

    # Fill Jug2

    jug2 = cap2

    print((jug1, jug2))

    # Pour Jug2 into Jug1

    jug1 = jug2

    jug2 = 0

    print((jug1, jug2))

    # Fill Jug2 again

    jug2 = cap2

    print((jug1, jug2))

    # Pour Jug2 into Jug1 until Jug1 is full

    transfer = cap1 - jug1

    jug1 = cap1

    jug2 = jug2 - transfer

    print((jug1, jug2))
```

```

# Empty Jug1

jug1 = 0

print((jug1, jug2))

# Transfer remaining water from Jug2 to Jug1

jug1 = jug2

jug2 = 0

print((jug1, jug2))

# Fill Jug2

jug2 = cap2

print((jug1, jug2))

# Pour Jug2 into Jug1

transfer = cap1 - jug1

jug1 = jug1 + transfer

jug2 = jug2 - transfer

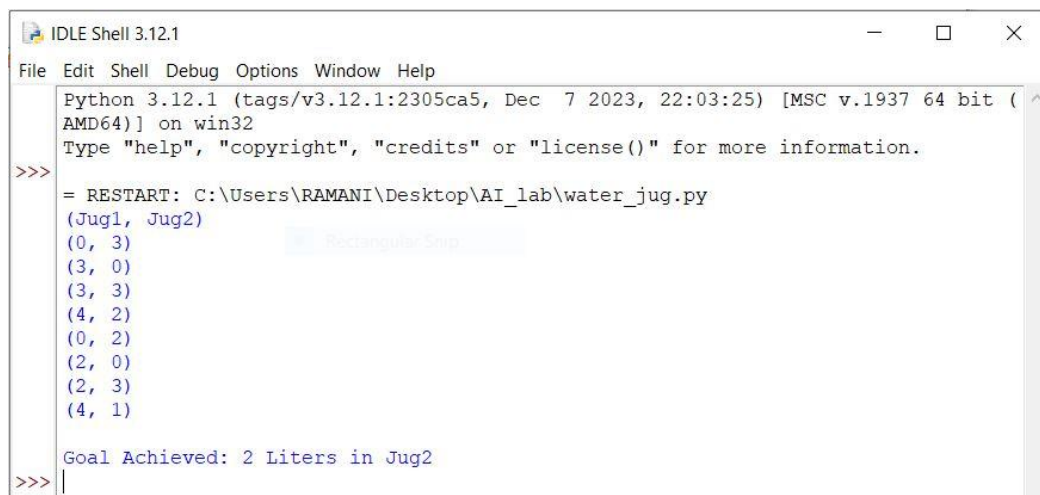
print((jug1, jug2))

print("\nGoal Achieved: 2 Liters in Jug2")

water_jug()

```

Output



```

IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\RAMANI\Desktop\AI_lab\water_jug.py
(Jug1, Jug2)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
(0, 2)
(2, 0)
(2, 3)
(4, 1)
Goal Achieved: 2 Liters in Jug2
>>>

```

Result

Thus, the Water Jug Problem was solved successfully and the desired quantity of water was obtained.

Experiment 4: Crypt Arithmetic Problem

Aim

To solve the Crypt Arithmetic Problem ($\text{SEND} + \text{MORE} = \text{MONEY}$) using Artificial Intelligence techniques.

Algorithm

1. Identify all unique letters in the given expression.
2. Assign unique digits to each letter.
3. Ensure that leading letters are not assigned zero.
4. Generate all possible digit combinations.
5. Check whether the arithmetic equation is satisfied.
6. If the equation is satisfied, display the solution.
7. Otherwise continue searching until a valid solution is found.

Program

```
from itertools import permutations

letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')

for p in permutations(range(10), 8):

    S, E, N, D, M, O, R, Y = p

    # Leading digits cannot be zero

    if S == 0 or M == 0:

        continue

    send = 1000*S + 100*E + 10*N + D

    more = 1000*M + 100*O + 10*R + E

    money = 10000*M + 1000*O + 100*N + 10*E + Y

    if send + more == money:

        print("Solution Found:")
```



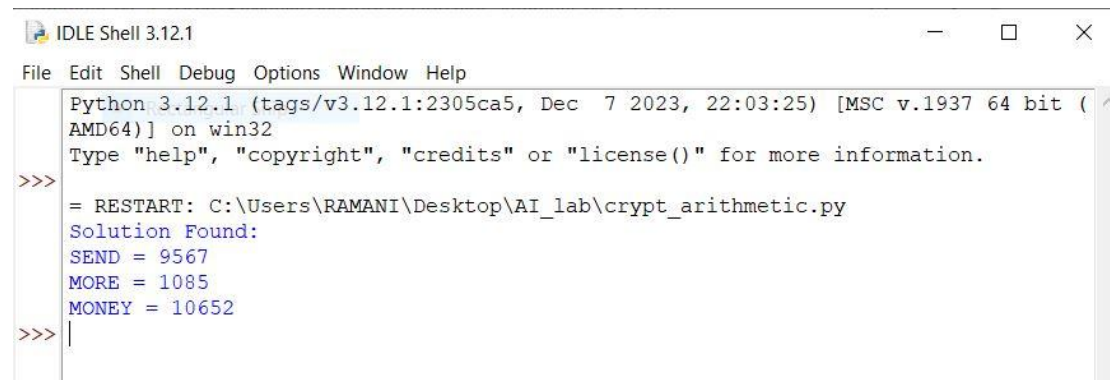
```
print("SEND =", send)

print("MORE =", more)

print("MONEY =", money)
```

Break

Output



```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\RAMANI\Desktop\AI_lab\crypt_arithmetic.py
Solution Found:
SEND = 9567
MORE = 1085
MONEY = 10652
>>> |
```

Result

Thus, the Crypt Arithmetic Problem was solved successfully and the correct digit assignment was obtained.

Experiment 5: Missionaries and Cannibals Problem

Aim

To solve the Missionaries and Cannibals Problem using Breadth First Search (BFS).

Algorithm

1. Represent the initial state as three missionaries and three cannibals on the left bank.
2. Define all possible boat movements.
3. Check whether a state is valid.
4. Generate successor states.
5. Use BFS to explore states level by level.
6. Avoid revisiting already explored states.
7. Continue until the goal state is reached.
8. Display the solution path.

Program

```
from collections import deque
```

```
# Initial state: (Missionaries Left, Cannibals Left, Boat Side)
```

```
start = (3, 3, 1)
```

```
# Goal state
```

```
goal = (0, 0, 0)
```

```
# Possible boat moves
```

```
moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
```

```
def is_valid(m, c):
```

```
    # Check limits
```

```
    if m < 0 or c < 0 or m > 3 or c > 3:
```

```
        return False
```

```
    # Left bank condition
```

```
    if m > 0 and m < c:
```

```
        return False
```

```
    # Right bank condition
```

```
    mr = 3 - m
```

```
    cr = 3 - c
```

```
    if mr > 0 and mr < cr:
```

```
        return False
```

```
    return True
```

```
def bfs():
```

```
    queue = deque([(start, [])])
```

```
    visited = set()
```

```
    while queue:
```

```
        state, path = queue.popleft()
```

```

    if state == goal:

        return path + [state]

    if state in visited:

        continue

    visited.add(state)

    m, c, boat = state

    for dm, dc in moves:

        if boat == 1: # Boat on left side

            new_state = (m - dm, c - dc, 0)

        else:         # Boat on right side

            new_state = (m + dm, c + dc, 1)

        nm, nc, _ = new_state

        if is_valid(nm, nc):

            queue.append((new_state, path + [state]))

    return None

solution = bfs()

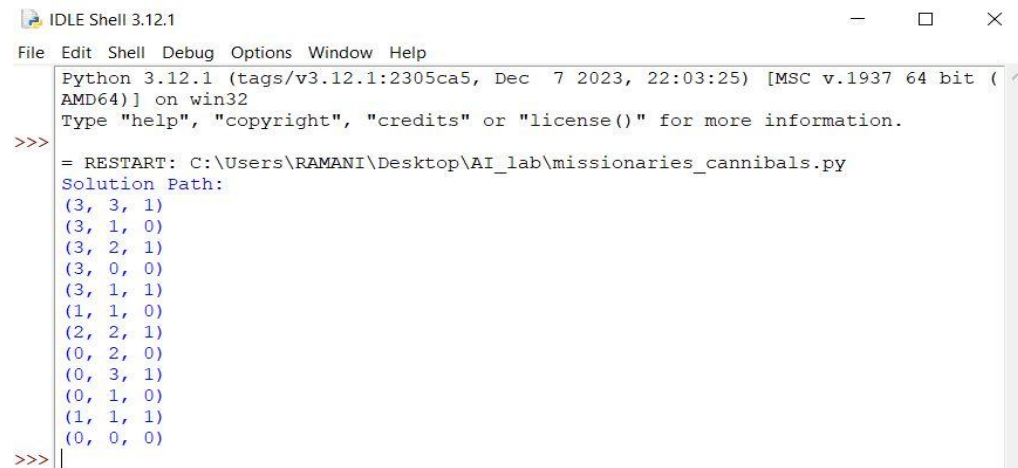
print("Solution Path:")

for step in solution:

    print(step)

```

Output



```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\RAMANI\Desktop\AI_lab\missionaries_cannibals.py
Solution Path:
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
>>> |
```

Result

Thus, the Missionaries and Cannibals Problem was solved successfully using Breadth First Search (BFS).

Experiment 6 : Vacuum cleaner problem

Aim

To implement the Vacuum Cleaner Problem using a Simple Reflex Agent.

Algorithm

1. Start with two rooms A and B.
2. Check the current room status.
3. If the room is dirty, clean it.
4. Move to the next room.
5. Repeat the process until all rooms are clean.
6. Display the final state.

Program

```
# Vacuum Cleaner Problem

# Room A and Room B status

rooms = {

    'A': 'Dirty',

    'B': 'Dirty'

}
```

```
# Vacuum starts at Room A

location = 'A'

print("Initial State:")

print("Room A =", rooms['A'])

print("Room B =", rooms['B'])

print()

# Cleaning process

while 'Dirty' in rooms.values():

    if rooms[location] == 'Dirty':

        print("Vacuum is in Room", location)

        print("Cleaning Room", location)

        rooms[location] = 'Clean'

    # Move to next room

    if location == 'A':

        location = 'B'

    else:

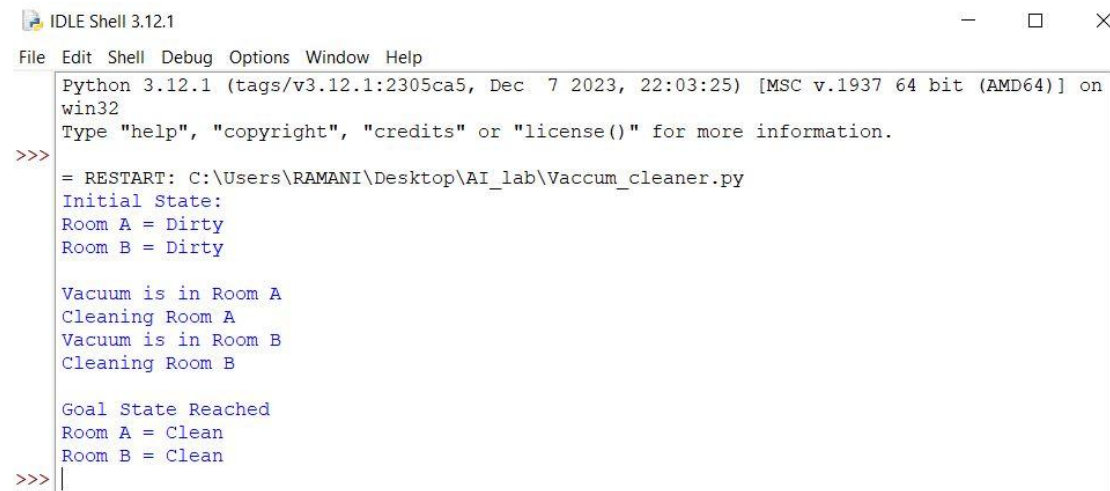
        location = 'A'

print("\nGoal State Reached")

print("Room A =", rooms['A'])

print("Room B =", rooms['B'])
```

Output



```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:\Users\RAMANI\Desktop\AI_lab\Vaccum_cleaner.py
Initial State:
Room A = Dirty
Room B = Dirty

Vacuum is in Room A
Cleaning Room A
Vacuum is in Room B
Cleaning Room B

Goal State Reached
Room A = Clean
Room B = Clean
>>> |
```

Result

Thus, the Vacuum Cleaner Problem was implemented successfully and all rooms were cleaned using a Simple Reflex Agent.